

第 04 讲：从单轮代码题到 Repo Issue

Coding Agent Notes (June 2026)

1 核心图景

从 HumanEval 到 SWE-bench 的变化，不只是题目越来越难。更准确地说，benchmark 改变了“任务契约”：给模型什么输入，要求它产出什么东西，哪些上下文需要自己找，什么算正确，失败以后能否从环境里得到反馈。



单轮代码题通常把边界提前切好：函数签名在哪里、要实现什么功能、用哪些测试判断正确，几乎都写在 prompt 里。repo issue 则不同。问题描述可能只说某个行为错了，真正需要修改的文件在仓库深处；修复可能只改十几行，但必须保持已有 API、风格和测试不被破坏。

Task difficulty ladder = prompt scope + context discovery + artifact type + oracle strength + state change.

所以，本讲的主线不是给这些 benchmark 排名，而是解释它们各自暴露了 coding agent 的哪一层能力。HumanEval/MBPP 让我们看到局部程序合成；APPS 让我们看到算法题式完整程序；CoderEval 让我们看到项目函数依赖；RepoBench 让我们看到跨文件上下文选择；SWE-bench 则把任务对象变成真实仓库状态和 patch。

2 任务契约的五個维度

每个 coding benchmark 都隐含一份契约。读分数之前，先要读这份契约。



Prompt。

prompt 决定问题如何被描述。HumanEval 的 prompt 是函数签名和 docstring；MBPP 是简短自然语言任务和测试；APPS 是完整题面；SWE-bench 是 GitHub issue 文本。越靠后，prompt 越像真实用户请求，也越可能含糊、遗漏、带截图、带错误栈、带上下文假设。

Context。

context 决定模型需要看到什么才能行动。单函数任务的 context 基本已经在 prompt 里；CoderEval 的函数可能依赖类、文件、项目 API；RepoBench 把检索跨文件片段变成独立任务；SWE-bench 中，相关文件可能只是几千个文件里的很小一部分。

Artifact.

artifact 是模型要交付的东西。早期任务交付一个函数体或程序；repo-level completion 交付下一行；SWE-bench 交付 patch。artifact 变化很关键：patch 必须能 apply，必须和已有代码融合，必须避免破坏旧行为。

Oracle.

oracle 决定什么叫正确。HumanEval/MBPP/APPS 主要用单题测试；CoderEval 需要在项目环境中运行目标函数测试；SWE-bench 用仓库测试验证 issue 是否解决。oracle 越接近真实工程，运行成本越高，噪声和环境问题也越多。

Failure Type.

失败类型也会变。单函数任务常见失败是逻辑错、边界情况漏掉；repo issue 可能是找错文件、改错抽象层、patch 格式坏、环境装不起来、测试没覆盖真实质量，或修复当前问题但引入 regression。

3 HumanEval 与 MBPP：局部正确性

HumanEval 的核心贡献是把代码生成评测从“像不像参考答案”推向“能不能通过测试”。它包含 164 个手写 Python 问题，每题有函数签名、docstring 和若干 unit tests。模型生成函数体，正确性由测试判断，并用 $\text{pass}@k$ 衡量多次采样中至少一个样本正确的概率。

这个设置非常干净。函数边界已经给定，问题规模小，评价信号明确。对于研究代码模型来说，这很有价值：如果一个模型连局部函数都写不稳，就很难指望它完成更长的软件工程任务。

但也正因为它干净，它主动拿掉了很多真实开发问题。模型不需要决定该读哪个文件，不需要理解项目约定，不需要处理依赖版本，不需要解释失败日志，也不需要维护已有行为。HumanEval 测的是“在边界已知时能否写对局部程序”。

MBPP 把问题扩展到 974 个 entry-level Python 任务。它仍然是短程序合成，但 prompt 更像自然语言需求，通常配有测试样例。这里出现了一个重要现象：测试并不总能完整表达意图。一个程序可能通过给定测试，但在未覆盖 corner case 上行为错误。

Insight. 单函数 benchmark 的价值是隔离 synthesis 能力；它的局限也是隔离得太干净。它能告诉我们模型是否会写局部程序，但不能告诉我们模型是否会在一个现存系统里做正确修改。

Benchmark	任务对象	被主动简化掉的内容
HumanEval	docstring 到函数体	上下文选择、项目依赖、patch 集成
MBPP	自然语言到短程序	真实需求澄清、测试覆盖不足、仓库状态

这两个 benchmark 仍然应该保留在能力谱系里。原因不是它们最接近真实工程，而是它们能提供一个可控底座：模型是否理解函数规格，是否能生成语法正确、语义基本正确的代码，是否能通过 executable oracle。它们像单元测试，不像系统测试。

3.1 $\text{pass}@k$ 到底在测什么

HumanEval 让 $\text{pass}@k$ 成为代码模型评测中的常用读法。直觉上， $\text{pass}@1$ 是“一次生成能否写对”， $\text{pass}@10$ 或 $\text{pass}@100$ 是“多采样以后有没有一个答案写对”。这很适合单函数任务，因为每个候选函数都可以独立运行测试，评估成本较低。

但 $\text{pass}@k$ 并不是纯粹的“模型理解力”。它同时包含三个变量：模型分布中正确解的密度、采样策略能否覆盖这些解、以及是否有 oracle 能从多个候选里挑出正确答案。对于 HumanEval，这三个变量相对容易控制；对于 repo issue，它们会变得复杂得多。

读法	单函数任务	Repo issue 任务
多采样	多个函数体候选	多个 patch、多个上下文、多个轨迹
选择依据	单题 tests	仓库测试、日志、patch apply、回归风险
成本	较低，执行快	高，环境启动和测试昂贵
风险	过拟合弱测试	过拟合 issue tests 或破坏旧行为

因此，看到一个 repo-level 系统用多候选、重排、验证时，不应该简单把它理解为“模型本身 $\text{pass}@k$ 高”。更准确地说，它是在用 harness 把多个可能的 patch 变成一个搜索问题。模型负责生成候选，环境负责筛掉明显错误，验证策略负责决定哪个候选可以交付。

3.2 测试不是完整规格

MBPP 的一个重要提醒是：测试样例可以帮助模型理解需求，但测试不是需求本身。一个程序可能通过给定测试，却在没有覆盖的输入上失败。单函数任务中，这通常表现为漏掉 corner case；软件工程任务中，它会升级成更严重的问题：修改可能通过 issue 相关测试，却破坏另一个模块、另一个平台或另一个用户路径。

这就是为什么“能通过测试”要和“测试覆盖了什么”一起读。测试是可执行信号，它让评测变得客观；但测试也是一个投影，它只观察行为空间的一部分。越真实的任务，行为空间越大，测试的盲区也越值得警惕。

Insight. unit tests 给了代码评测一个可执行 oracle，但 oracle 的强度取决于测试覆盖。越靠近真实软件工程，越不能把“通过测试”简单等同于“修复质量完整”。

4 APPS：算法难度不等于工程难度

APPS 往上走了一步。它包含 10,000 个 coding challenge，从 introductory 到 interview 再到 competition level，配有超过 130,000 个测试用例和大量参考解。模型需要读自然语言题面，生成完整 Python 程序，并通过测试。

APPS 的难点是真实的：题面更长，输入输出格式更复杂，算法要求更高，有些题需要动态规划、图算法、数据结构或细致 corner case 处理。相比 HumanEval，APPS 更像在线判题系统里的编程题。

但是，APPS 的“难”主要是算法题难，不是软件工程难。题目通常自成一体，目标是从规格写出一个程序；模型不需要理解一个已经存在的代码库，也不需要已在已有抽象层里插入最小改动。

维度	APPS	Repo Issue
主要压力	算法设计、题面理解、IO 格式	上下文定位、依赖关系、patch 集成
代码环境	多数自包含	依赖现有仓库和 runtime
正确性	通过题目测试	解决 issue 且不破坏旧行为
典型失败	算法错、复杂度高、边界漏	找错文件、改错层、regression

这一区分非常重要。一个模型在 APPS 上强，说明它有较强的算法推理和程序构造能力；但这不自动推出它能修 repo issue。repo issue 往往不是“想出一个算法”，而是“理解这个系统为什么在这里坏了，并用符合项目约定的方式修掉”。

4.1 三种不同的难

coding benchmark 里经常把“难”混在一起说，但至少有三种难度需要分开。

算法难。 模型要找到正确算法、满足复杂度、处理输入输出格式。APPS 的 competition 题主要是这类难度。它要求强推理，但任务通常还是封闭的：题目给出完整规格，答案是一个自包含程序。

上下文难。 模型要知道哪些外部信息相关。CoderEval 里的函数可能依赖当前类、当前文件或项目 helper；RepoBench 里的下一行可能依赖另一个文件中的定义。这类难度不是“想出算法”，而是“找对并使用周围代码”。

状态难。 模型要改变一个已有系统，同时保留旧行为。SWE-bench-style task 是这类难度。patch 不是孤立答案，而是对仓库状态的变更；变更以后，测试、依赖、导入路径、风格和后续维护都会产生影响。

难度类型	代表 benchmark	典型能力
算法难	APPS	题面理解、算法选择、复杂度控制
上下文难	CoderEval / RepoBench	依赖识别、跨文件检索、API 使用
状态难	SWE-bench	patch 集成、测试反馈、回归控制

这三种难度可以叠加，但不能互相替代。一个模型可以很会解算法题，却不擅长在陌生仓库里找 helper；也可以很会补局部代码，却在 issue repair 中因为找错文件而失败。把难度拆开，才能读懂不同 benchmark 的互补性。

5 CoderEval：函数开始依赖项目

CoderEval 把任务带进真实项目，但还没有完全进入 issue repair。它关注 pragmatic code generation：目标仍然是生成函数，但函数可能不是 standalone。论文指出，流行开源项目中大量函数依赖外部上下文；只测 standalone function 会高估模型在真实项目中的可用性。

CoderEval 包含 230 个 Python 问题和 230 个 Java 问题，来自真实开源项目。它把函数按 runnable scope 分类：self-contained、standard-library runnable、public-library runnable、class-runnable、file-runnable、project-runnable。这个分类本质上是在问：为了让这个函数工作，模型必须知道多远的上下文？

Runnable level	依赖范围	对模型意味着什么
self-contained	函数内部	类似 HumanEval
slib-runnable	标准库	需要标准 API 知识
plib-runnable	第三方库	需要库语义和版本假设
class-runnable	当前类	需要字段、方法、类状态
file-runnable	当前文件	需要文件级变量和辅助函数
project-runnable	项目内部	需要跨文件和项目约定

CoderEval 的核心 insight 是：函数看起来仍然是函数，但它的正确性已经不再只由函数体决定。它

可能需要调用项目 API，遵守类不变量，使用已有 helper，返回项目约定的数据结构。模型如果只生成局部看似合理的代码，就容易在项目环境里失败。

这一步是从“代码题”到“工程代码”的关键桥梁。任务输出仍然不是 patch，也不要求理解用户 issue，但模型必须开始尊重 surrounding code。换句话说，context 不再是背景材料，而是 correctness 的一部分。

5.1 为什么 non-standalone function 会改变正确性

在 standalone function 里，函数体是主要世界。只要输入输出符合测试，内部如何写通常不重要。到了 non-standalone function，函数体只是项目的一部分。它可能要读取类字段，调用同文件 helper，返回项目自定义对象，遵守异常处理约定，或使用某个第三方库的特定版本。

这会带来一个非显然的后果：局部看起来正确的代码，可能在项目里不正确。例如模型自己重新实现一个小工具函数，在单独测试里能跑通，但项目里已有 helper 处理了日志、类型转换或兼容性；模型绕过 helper，就可能产生风格不一致、边界行为不同或未来维护困难的代码。

上下文类型	模型容易忽略的东西	可能后果
类状态	字段含义、初始化时机	修复只在部分实例上有效
文件 helper	复用函数、常量、错误类型	重复实现或行为不一致
项目 API	内部抽象层、兼容接口	破坏调用方假设
第三方库	版本、返回类型、副作用	运行时失败或隐性 bug

所以 CoderEval 的意义不只是“比 HumanEval 多了真实项目”。它把正确性的单位从函数内部推到函数与周围代码的关系。这个关系正是 repo issue repair 的基础：真实修复很少是凭空写新代码，而是在已有抽象层里做最小、合适、可维护的改变。

5.2 runnable level 是上下文半径

CoderEval 的 runnable level 可以理解为“上下文半径”。self-contained 半径最小，project-runable 半径最大。半径越大，模型越需要知道外部定义；但半径大不一定意味着代码本身更长或算法更复杂。一个 project-runable 函数可能只有几行，难点却在于那几行必须调用正确的项目接口。

这对 coding agent 的启发很直接：不要只按 token 数估计任务难度。真正的难度往往来自上下文半径。一个短 issue 如果牵涉项目约定、跨文件调用或隐性不变量，可能比一个长但自包含的算法题更难。

6 RepoBench：上下文选择成为任务

RepoBench 进一步把 repository-level completion 拆成三个子任务：检索跨文件片段的 RepoBench-R，给定上下文预测代码的 RepoBench-C，以及把检索和补全连起来的 RepoBench-P。这个拆法非常适合理解 coding agent，因为它把“找到正确上下文”从隐含步骤变成显式评价对象。

在单文件补全里，模型只要看当前文件前文；在真实仓库里，下一行代码可能取决于另一个文件里的类定义、工具函数、常量、导入路径或调用约定。RepoBench 关注的正是这种跨文件依赖。

Repository completion = retrieve cross-file context + combine with in-file context + predict local code.

RepoBench 还说明一个常被低估的问题：长上下文不是自动答案。模型能塞进更多 token，不等于

它知道哪些 token 重要。检索器、repo map、路径名、import graph 和代码片段排序都会影响最终补全质量。到了 coding agent，这个问题会进一步扩大：agent 不只是补下一行，而是要决定下一步读哪个文件、运行哪个命令、修改哪里。

因此，RepoBench 在任务阶梯中的位置很清楚。它还不是 issue repair，因为输出不是 patch，也不需要运行完整仓库测试。但它把 repo-level task 中最核心的中间变量暴露出来：context selection。

6.1 检索错误会如何传播

RepoBench 的检索-补全拆分很有价值，因为它让我们看到错误传播。第一步如果没找对 cross-file snippet，第二步的补全模型再强，也可能只能在错误上下文里生成一个局部合理但项目不合适的答案。

在 coding agent 里，这种传播会更明显。agent 第一次读错文件，后续 plan 就会围绕错误假设展开；生成 patch 时可能修改相邻但不相关的代码；测试失败后，模型又可能解释成另一个错误原因。长任务里的错误不是独立采样，而会沿着轨迹累积。

阶段	局部错误	后续影响
检索	漏掉关键定义	生成代码使用错误 API
排序	相关片段位置太靠后	长上下文中被模型忽略
压缩	删除错误栈或调用点	误判 bug 原因
补全	只修表面症状	测试过少时形成假阳性

这也是为什么 repo-level task 不能只靠“更大上下文窗口”解决。上下文窗口解决容量问题，但不解决选择问题。选择问题需要检索、结构化 repo map、AST/LSP 信号、路径名、测试反馈和历史轨迹共同参与。

7 SWE-bench: 任务对象变成仓库状态

SWE-bench 把输入换成真实 GitHub issue 和代码库 snapshot，把输出换成 patch，把 oracle 换成仓库测试。它包含 2,294 个问题，来自 12 个 Python 仓库的真实 issue 与对应 PR。原始论文报告中，Claude 2 在基线设置下只解决了 1.96% 的问题，这个数字本身就说明任务已经远超传统代码生成。



这里的关键变化是：模型不再是“写一个程序”，而是“改变一个系统状态”。它需要理解 issue，定位相关代码，生成 patch，把 patch apply 到仓库，运行测试，再根据结果判断是否解决。即使最终只改一个文件、一个函数，发现这一步也可能需要阅读错误栈、搜索符号、理解项目 API、排除无关文件。

维度	单函数任务	SWE-bench-style issue
输入	函数签名、docstring、样例	issue 文本、仓库 snapshot
上下文	已经给定	需要检索、筛选、压缩
输出	函数体或程序	patch / diff
验证	单题 unit tests	仓库测试、issue tests、回归检查
失败	局部逻辑错误	环境、上下文、patch、regression 共同作用

SWE-bench 也让 harness 变得不可避免。要跑这种任务，系统必须能 checkout repo、安装依赖、应用 patch、执行测试、处理 timeout、解析日志、记录轨迹。模型权重只是其中一层；没有运行制度，repo issue repair 就没有稳定的可验证信号。

这也是为什么 repo-level benchmark 不能用单函数 benchmark 的直觉来读。单函数任务的失败常常能归因到模型没有写对逻辑；SWE-bench 的失败可能来自检索器漏掉关键文件，context 太长导致注意力分散，patch 格式错误，环境装不起来，或者测试只覆盖了一部分行为。

7.1 issue repair 为什么是状态改变

repo issue repair 的输入不是一个数学题，而是一个系统状态：代码库在某个 commit 上，依赖有特定版本，测试有特定行为，issue 描述了当前状态和期望状态之间的差距。patch 的作用是把旧状态变成新状态。

这个视角比“生成代码”更准确。生成代码只关注答案文本；状态改变关注 patch 应用以后系统发生什么。一个 patch 可以语法正确、局部测试通过，但仍然不是好状态：它可能绕过原有抽象，改变公共 API，增加隐藏复杂度，或让另一个模块的假设失效。

Repo issue repair = current codebase state + desired behavior + patch transition + executable validation.

从这个角度看，测试是对状态转移的抽样检查。它不能完全证明新状态正确，但可以排除大量明显错误。agent 的价值在于把这套状态转移过程自动化：读 issue，找上下文，提出 patch，运行验证，根据失败反馈继续调整。

7.2 patch 比完整文件更接近工程交付

SWE-bench 使用 patch/diff 作为核心输出，这也很有工程含义。软件工程里的交付通常不是“重写整个文件”，而是一个可审查的修改。patch 能暴露改了哪里、加了什么、删了什么，也能被 git、review、CI 和测试系统处理。

但 patch 也引入了格式和定位问题。模型需要生成能 apply 的 diff，需要保留上下文行，需要避免误改无关区域。一个语义上接近正确的修复，如果 diff 格式坏了，也无法进入验证阶段。对 coding agent 来说，这说明 action interface 很重要：让模型直接写 patch、用编辑工具修改文件、还是通过 AST 操作改代码，会影响失败类型。

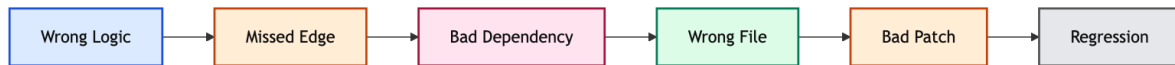
7.3 repo issue 的 oracle 是多层的

单函数任务里，oracle 通常是同一组 tests。repo issue 中，oracle 更像多层过滤。首先 patch 要能应用；然后代码要能安装或运行；然后 issue 相关测试要从失败变成通过；最后已有行为不能被破坏。任何一层失败，最终都不是成功。

这里最值得记住的是：测试通过只是最低交付条件，不是唯一质量条件。真实工程还关心可维护性、风格一致、性能、安全、兼容性和 review 可接受度。benchmark 必须选择可自动化的 oracle，但读者要知道自动化 oracle 只是工程质量的一部分。

8 失败类型如何变化

任务阶梯越往后，失败越不像一个局部错误，越像系统失配。



失败类型	常见位置	说明
Wrong logic	HumanEval / MBPP	代码语义不符合局部规格
Missed edge	MBPP / APPS	测试覆盖不足或 corner case 未处理
Bad dependency	CoderEval	没用对类、变量、库或项目 API
Wrong file	RepoBench / SWE-bench	没找到真正相关上下文
Bad patch	SWE-bench	diff 不能 apply 或修改位置不合适
Regression	SWE-bench	当前 issue 过了但旧行为被破坏

这张失败图说明一个实际工程判断：当 benchmark 往 repo issue 方向移动时，简单的 pass rate 变得不够解释性。一个 30% pass rate 的系统，如果失败类型清楚，可能比一个 35% 但失败不可诊断的系统更有研究价值。因为前者告诉我们应该改检索、改工具、改验证，还是改模型。

同样，这也解释了为什么单轮代码题不能被简单淘汰。它们仍然是检查局部代码能力的低成本单元测试；只是不能把它们当成完整软件工程能力的代表。

8.1 为什么平均分不够

平均 pass rate 把不同失败压成一个数字。这个数字便于排行榜，但不便于改系统。一个 agent 如果大量失败来自找不到文件，应该改 context pipeline；如果大量失败来自 patch 格式坏，应该改编辑接口；如果大量失败来自测试日志太长，应该改 observation compression；如果大量失败来自过早停止，应该改 loop policy。

因此，repo-level 评测更需要失败分解。分解后的结果可能不如单个分数漂亮，但更接近工程现实。好的系统报告应该告诉读者：失败主要发生在任务理解、上下文定位、编辑、验证、环境，还是停止策略。

失败主因	应该优先改	不应该误判为
找不到相关文件	retrieval / repo map	模型不会写代码
patch apply 失败	action interface	需求理解失败
测试日志误读	observation compression	算法能力不足
修复后 regression	validation policy	pass@1 太低
轨迹太早终止	loop / budget	base model 不够大

这张表是阅读 benchmark 结果时最实用的诊断框架。它提醒我们：越靠近真实工程，失败越可能是系统层失败，而不是某一个 token 预测错了。

9 如何读 benchmark 分数

读一个 coding benchmark 分数时，最少要问六个问题。

第一，任务 rung 是什么。

它是单函数、完整程序、项目函数、repo completion，还是 issue repair？不同 rung 的分数不能直接互相替代。

第二，上下文是给定还是检索。

如果 benchmark 给了 oracle context，它测的是生成能力；如果系统必须自己找 context，它还测检索、压缩和排序。

第三，输出 artifact 是什么。

函数体、整程序、下一行和 patch 的难点不同。patch 需要格式正确、位置正确、能和现有代码融合。

第四，oracle 有多强。

测试数量、隐藏测试、项目测试、回归测试都会影响分数含义。测试越弱，越容易出现“看似解决”的答案。

第五，环境是否可复现。

repo-level 任务需要依赖安装、运行脚本、文件权限和 timeout。环境不稳时，分数会混入 runtime 噪声。

第六，失败是否被分类。

只给平均分不够。定位失败、编辑失败、测试失败、环境失败、停止失败，对应完全不同的改进路线。

Insight. benchmark 分数不是单个数字，而是任务契约下的测量结果。越靠近 repo issue，越要同时读输入、上下文、artifact、oracle 和 harness。

10 把这些 benchmark 当作 curriculum

这些 benchmark 也可以被看成一条训练或评测 curriculum。它们不是互相替代，而是分层提供信号。

底层：局部程序合成。 HumanEval 和 MBPP 适合检查模型是否具备基本 synthesis 能力。这里的反馈快、任务短、失败可解释。它们适合作为低成本 sanity check。

中层：完整规格和项目上下文。 APPS 提供更强算法压力；CoderEval 和 RepoBench 提供项目上下文压力。它们让我们分开观察“会不会设计程序”和“会不会使用周围代码”。

高层：状态改变和环境验证。 SWE-bench-style task 把模型放进更接近工程的闭环里。这里不仅测生成，还测 context、tool、runtime、verification 和 failure recovery。

层级	适合回答的问题	不适合推出的结论
局部合成	模型会不会写小程序	能不能修真实仓库
算法挑战	模型会不会解复杂题	会不会遵守项目约定
项目上下文	模型会不会用外部依赖	会不会完整处理 issue 生命周期
repo issue	系统能否完成状态改变	纯模型权重到底多强

如果把它们当作 curriculum，最健康的评测报告不应该只给最高层分数。它应该同时报告底层局部能力、中层上下文能力和高层 repo repair 能力。这样我们才能知道系统强在哪里、短板在哪里，以及下一步该改模型还是改 harness。

11 最终 takeaway

从单轮代码题到 repo issue，可以压缩成三句话。

- HumanEval/MBPP/APPS 主要测局部或自包含程序生成；它们重要，但不能代表完整软件工程任务。

- CoderEval/RepoBench 把项目依赖和跨文件上下文推到前台，是从代码模型到 coding agent 的中间桥梁。
- SWE-bench-style issue repair 的本质是状态改变：找到上下文，生成 patch，运行验证，并避免 regression。

因此，一个 coding agent 如果只在单函数 benchmark 上强，还不能说明它会修真实仓库；但如果它连单函数 benchmark 都不稳，也很难在 repo issue 上可靠。正确读法是把这些 benchmark 看成一组分层测试：越底层越可控，越高层越接近真实工程，也越需要系统设计。

参考资料

- [1] [HumanEval](#). functional correctness for standalone Python functions
- [2] [MBPP](#). entry-level short Python programming tasks
- [3] [APPS](#). coding challenge benchmark with test cases
- [4] [CoderEval](#). pragmatic project-function code generation
- [5] [RepoBench](#). repository-level code completion and retrieval
- [6] [SWE-bench](#). GitHub issue repair with patch validation